

DAILY ENGINEERING REPORT

iwmQT Trading Terminal

What shipped today, explained plainly - a new two-way data feed, proof the system holds up under real load, a mistake we caught and fixed, and the backup/redundancy picture.

Prepared for: Senior Review

Date: 31 August 2026

Branch: feature/dev

1

NEW FEATURE SHIPPED

2

SAFETY NETS ADDED

1

MISTAKE CAUGHT & FIXED

18

ROWS RECOVERED, BYTE-FOR-BYTE

The short version

Four things happened today. In plain terms:

- **A new pipe was built between our platform and outside algo/strategist systems** - they can now send us live signals, and we can now send commands back to them, in real time.
- **We proved the system doesn't fall over under real traffic** - an automated check now runs on every code change, and we deliberately hammered the order-placement endpoint far past its advertised limit to see what actually breaks first.
- **We made a mistake, caught it ourselves, fixed the real data, and closed the door on it happening again** - explained in full in Section 4, because it's more useful to show than to bury.
- **Backups are now actually running on a schedule**, not just sitting there as a script - and the plan for full database redundancy is written down, waiting on one business decision.

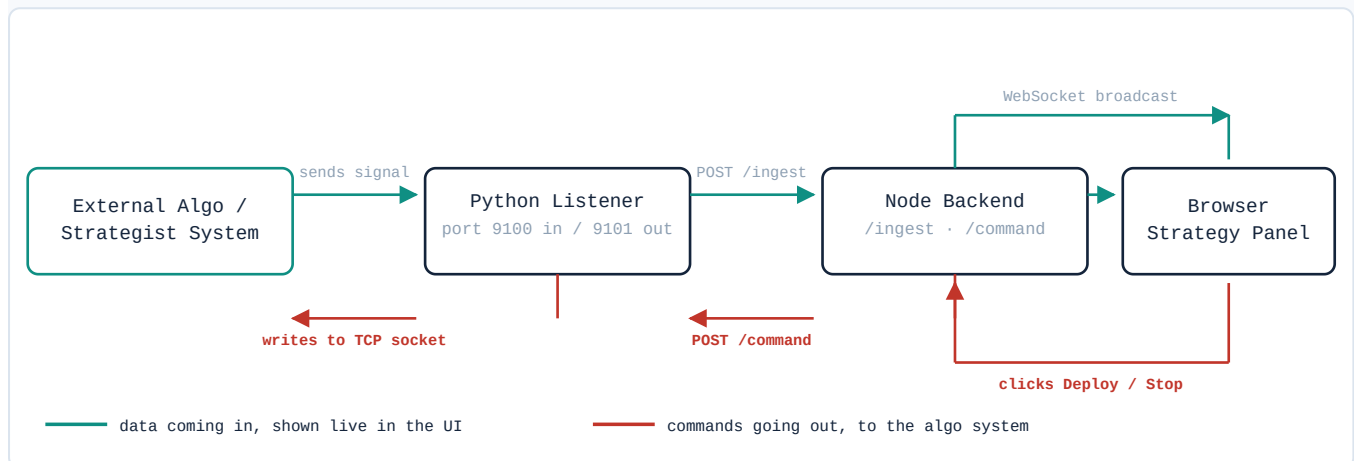
NET EFFECT

Nothing about how traders, PMs, or RMS admins use the platform changed today. Everything below is either a brand-new capability (the algo feed) or the platform getting more trustworthy in the background.

A two-way pipe to strategists and algo systems

Your ask was specific: when an outside strategist or algo system sends us an update, we should show it - and we should be able to send messages back to that system too, not just watch. Both directions are built and tested.

How it actually flows



Two independent paths sharing the same three middle hops - data flows in (teal) and commands flow out (red), each verified end-to-end with a live test message.

What was actually tested, not just written

- A real TCP client sent a JSON signal in - it showed up correctly in the panel's live feed.
- A trader's Deploy/Stop click was sent all the way through to a connected test client, which actually received it.
- The shared secret between the Python listener and the backend was moved out of shell commands and into a local `.env` file, so it never lands in anyone's terminal history.

SCOPE, HONESTLY

This only feeds the Strategy panel right now - it doesn't touch orders, positions, or anything else in the platform. If a similar feed is needed somewhere else later, it would need its own version of this same pattern.

These controls are supposed to work together, and today's test proved they do - a flood of orders gets stopped fast, from more than one direction, with no crashes and no slowdown (response times stayed under 10 milliseconds even at 700 orders/sec).

What went wrong, and what it means

While building the load test in Section 3, its cleanup step deleted 2 real orders (and their related records) that belonged to a real trader account - not test data. This is reported in full because it's the kind of thing that needs to be visible, not because it was left unresolved.

What happened

Before

Test reused a real trader's account

Cleanup: delete everything for that user
→ wiped 2 real orders + history

After

Test always creates a fresh, disposable account

Cleanup: only that disposable account
→ real data physically out of reach

Hard guard added: cleanup now refuses to run at all if any account in scope isn't one it created itself - this mistake is now structurally impossible.

The fix isn't "be more careful" - it's that the dangerous option no longer exists in the code.

How it was caught and fixed, same session

1. A row count didn't add up during routine verification - checked it immediately instead of assuming it was fine.
2. Confirmed 2 real orders (and their fills, history, and audit trail) had been deleted.
3. Restored all 18 affected rows from the nightly backup, byte-for-byte - same IDs, same timestamps, same values, verified against the original.
4. Rewrote the test tool so it can never touch a real account again, with a hard check that refuses to run if it ever tries.

WHY THIS BELONGS IN THE REPORT

This is also the strongest real-world proof yet that the backup system actually works - not a structural check this time, but an actual recovery of actual deleted data, verified to match exactly.

What's real now, what's still a plan

ITEM	STATUS	WHAT THAT MEANS
Nightly backup	RUNNING	Runs automatically every night at 2am, keeps 14 days, confirmed on the schedule itself (not just written to run)
Restore capability	PROVEN	Actually used to recover real data today (Section 4) - the strongest possible proof it works
Redundancy (zero-downtime failover)	DOCUMENTED	Written up in full, but not built - it depends on a decision that hasn't been made yet (see below)

WHY REDUNDANCY IS WAITING, NOT STALLED

Backup answers "can we get the data back." Redundancy answers "does trading keep working if the database server dies right now." That second one depends entirely on where this eventually gets deployed - a managed cloud database (AWS, GCP, Azure) gets you this as a setting, almost for free; self-hosting means building it by hand with dedicated tooling. Building the self-hosted version now would likely be thrown away once that decision is made, so the plan is written and ready, not built blind.

What's left from here

- **Uptime alerting** - notify someone the instant the backend goes down. Blocked on the same thing as redundancy: there's no public URL yet for an external monitor to watch, since nothing is deployed publicly.
- **Secrets management** - move credentials out of local `.env` files into a real secrets manager, once a deployment target is chosen.
- **The deployment decision itself** - this is now the one thing blocking the most remaining engineering work. Everything above that's "documented, not built" turns into a short, concrete task the moment it's made.